

Blockchain-Based Software-Defined Industrial Internet of Things: A Dueling Deep Q -Learning Approach

Chao Qiu, *Graduate Student Member, IEEE*, F. Richard Yu^{ID}, *Fellow, IEEE*, Haipeng Yao^{ID}, *Member, IEEE*, Chunxiao Jiang, *Senior Member, IEEE*, Fangmin Xu, and Chenglin Zhao

Abstract—With the developments of communication technologies and smart manufacturing, Industrial Internet of Things (IIoT) has emerged. Software-defined networking (SDN), a promising paradigm shift, has provided a viable way to manage IIoT dynamically, called software-defined IIoT (SDIIoT). In SDIIoT, lots of data and flows are generated by industrial devices, where a physically distributed but logically centralized control plane is necessary. However, one of the most intractable problems is how to reach consensus among multiple controllers under complex industrial environments. In this paper, we propose a blockchain (BC)-based consensus protocol in SDIIoT, along with detailed consensus steps and theoretical analysis, where BC works as a trusted third party to collect and synchronize network-wide views between different SDN controllers. Specially, it is a permissioned BC. In order to improve the throughput of this BC-based SDIIoT, we jointly consider the trust features of BC nodes and controllers, as well as the computational capability of the BC system. Accordingly, we formulate view change, access selection, and computational resources allocation as a joint optimization problem. We describe this problem as a Markov decision process by defining state space, action space, and reward function. Due to the fact that it is difficult to solve this joint problem by traditional methods, we propose a novel dueling deep Q -learning approach. Simulation results are presented to show the effectiveness of our proposed scheme.

Index Terms—Blockchain (BC), dueling deep Q -learning (DQL), Industrial Internet of Things (IIoT), multiple controllers, software-defined networking (SDN).

Manuscript received June 8, 2018; revised August 28, 2018; accepted September 14, 2018. Date of publication September 20, 2018; date of current version June 19, 2019. This work was supported in part by the Key Program of the National Natural Science Foundation of China under Grant 61431008, in part by the BUPT Excellent Ph.D. Students Foundation under Grant 2015010100, and in part by the China Scholarship Council under Grant 201706470022. (Corresponding author: Haipeng Yao.)

C. Qiu is with the Key Laboratory of University Wireless Communication, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China.

F. R. Yu is with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada.

H. Yao is with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: yaohaipeng@bupt.edu.cn).

C. Jiang is with the Department of Aerospace Engineering, Tsinghua University, Beijing 100084, China.

F. Xu and C. Zhao are with the Key Laboratory of Universal Wireless Communication, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China.

Digital Object Identifier 10.1109/JIOT.2018.2871394

I. INTRODUCTION

RECENTLY, there are a growing number of applications that use Internet of Things (IoT) technologies in several industries. Industrial Internet of Things (IIoT) has emerged and attracted lots of attentions from industry and academia [1]. In order to meet the demands of high bandwidth, ubiquitous accessibility, and dynamic management, software-defined networking (SDN) [2] has been used in IIoT, called SDIIoT [3]. In addition, software-defined routing management, edge computing, flow scheduling, and energy harvesting have been researched in [4]–[7]. With the increasing number of industrial devices, more than one controller is employed in SDIIoT, known as distributed SDIIoT. How to reach consensus among multiple controllers is challenging in distributed SDIIoT.

Although some traditional methods enable to reach consensus among multiple controller instances, numerous nontrivial issues in the current consensus methods prevent SDIIoT from being used as a generic platform for different services and applications, including: 1) the extra overheads; 2) the poor safety and liveness properties; and 3) the limited scope of available network size. These challenges need to be broadly tackled by comprehensive research efforts.

Recently, Blockchain (BC) [8] has been emerged as a novel technique, which can be used to address the above challenges. BC is a distributed ledger to record transactions, and provide trustworthy services to a group of nodes without central authority [9]. For the distributed SDIIoT, BC can act as a trusted third and “out-of-band” party to collect and synchronize network-wide views (e.g., network events, network topology, OpenFlow commands, etc.) between different SDN controllers safely, dependably, and traceably. In general, there are two kinds of BC [10], including permissionless BC and permissioned BC. In permissionless BC, enrollment is open to anyone, and nodes enable to join and leave dynamically and frequently, using Nakamoto consensus protocol and coupled to cryptocurrency, such as proof of work in Bitcoin [8] and proof of stake in Ethereum [11]. All BC participants (miners) contribute their CPU power to work on an extra hard task, and only the winner of them enables to propose a block and synchronize it with others. Thus, lots of resources and time are imposed in the permissionless BC. Moreover, another BC, the permissioned BC, uses *Byzantine* fault tolerance (BFT) consensus protocol. It employs

state machine replication mechanism to deal with *Byzantine* nodes that are subverted by adversaries and against the common goal of reaching consensus maliciously [12], such as practical BFT (PBFT) [13] and Paxos [14]. They always operate in a partially trusted environment, such as Hyperledger Fabric [15]. Thus, the advantages of permissioned BC are low costs, low latency, and low band-intensive. Considering partial trust, limited communication time, high loads and narrow bandwidth in SDIIoT, as well the advantages of permissioned BC, we consider to use permissioned BC in this paper.

In this paper, we propose a BC-based consensus protocol in distributed SDIIoT, where BC works as a trusted third party to collect and synchronize network-wide views between different SDN controllers. Specifically, it is a permissioned BC. Due to the fact that the throughput of permissioned BC is constrained with respect to the performance of BFT consensus protocol, we jointly consider the trust features of BC nodes and controllers, as well as the computational capability of BC system, so as to further improve the throughput of BC. Accordingly, we formulate view change, access selection, and computational resources allocation as a joint optimization problem. We describe this problem as a Markov decision process by defining state space, action space, and reward function. Inspired by the works to achieve the optimal decisions by improved learning methods [16]–[18], and the fact that it is difficult to solve this joint problem by traditional methods, we use a novel dueling deep Q -learning (DQL) approach to solve this problem. A preliminary version of this paper appeared in [19]. The distinct contributions of this paper are as follows.

- 1) We propose a BC-based consensus protocol to simplify and secure the collection and synchronization of network views between different SDN controllers. And we give consensus steps in detail, along with theoretical analysis.
- 2) Due to the fact that lots of network views need to be collected and synchronized by BC system, we jointly consider the trust features of BC nodes and controllers, as well as computational capability of the BC system, so as to improve the throughput. Accordingly, we formulate view change, access selection, and computational resources allocation as a joint optimization problem. We describe it as a Markov decision process by defining state space, action space, and reward function.
- 3) In order to address this joint problem, we propose a novel dueling DQL approach to learn the optimal strategy. Simulation results with different system parameters are presented to show the effectiveness of the proposed scheme.

The rest of this paper is organized as follows. In Section II, we present some related works about SDIIoT and distributed SDN. Section IV describes the BC-based consensus protocol, followed by the system model in Section III. In Section V, we describe view change, access selection, and computational resources allocation as a Markov decision process. We then use a novel dueling DQL approach to solve this issue in Section VI. Simulation results are presented and discussed in Section VII. Finally, the conclusion and future works are given in Section VIII.

II. RELATED WORK

In this section, we briefly present the recent advances in SDIIoT and distributed SDN. Some challenges are discussed as well.

A. SDIIoT

There are a number of advances in IIoT, such as industrial wireless sensor networks, industrial big data, cloud computing, and fog computing. These emerging techniques bring the explosion of intelligent devices, industrial robots that are supported by wired or wireless networks. In order to meet the demands of high bandwidth, ubiquitous accessibility, and dynamic management, lots of works have employed SDN in IIoT. For example, Wan *et al.* [20] proposed a software-defined industrial network to manage manufacturing resources dynamically. Moness *et al.* [21] employed a hybrid software-defined approach in IIoT. Silva *et al.* [22] expanded OpenFlow protocol in industrial scenarios. Duan *et al.* [23] described a software-defined and virtual multiple networks control framework in IIoT.

As we can see that there is an emerging trend to employ SDN in IIoT. Due to the fact that there are lots of data and flows in SDIIoT, more than one controller is necessary, called distributed SDIIoT. One of the most intractable problems in distributed SDIIoT is how to reach consensus among multiple controllers. Whether or not traditional distributed SDN architectures are suitable in IIoT, and how to reach consensus among controllers need to be further researched. Therefore, we will present some traditional distributed SDN architectures, along with the corresponding consensus methods and their challenges.

B. Distributed SDN Architectures

Traditionally, there are lots of works to design distributed SDN architectures [24]. Based on the configuration methods between controllers and switches, we classify them as statically configured control architecture and dynamically configured control architecture [25].

1) *Statically Configured Control Architecture:* Tootoonchian and Ganjali [26] designed the first distributed SDN control plane for OpenFlow, called HyperFlow. It synchronized controllers' network-wide views (i.e., local controller instance's events) by a publish/subscribe messaging paradigm. Koponen *et al.* [27] considered a Onix architecture. They used network information base (NIB) to aggregate and share network-wide views.

2) *Dynamically Configured Control Architecture:* Statically configured control architectures lead to the uneven load distribution among controllers. Therefore, Berde *et al.* [28] designed an elastic and experimental distributed SDN control plane, called ONOS. The work in [29] and [30] presented a hierarchical SDN control architecture with the joint consideration of load balancing and energy consumption of multiple controllers.

As we can see from the above researches, one of the most important problems in distributed SDN architectures is how to reach consensus among multiple controller instances.

No matter which consensus protocols (e.g., publish/subscribe, slicing, NIB, controllers pool, etc.), lots of challenges are remained to be solved when they are employed in SDIIoT.

- 1) The traditional consensus protocols are implemented “in-band,” which leads to the extra overheads in each controller to weaken the inherent functions of controllers (e.g., routing decisions, network management, etc.). Especially, due to the limited capacities of industrial devices, this problem is more severe. Therefore, a third party and out-of-band consensus protocol is necessary in SDIIoT.
- 2) Safety and liveness properties are ignored in traditional works. As the brain of SDN, the failures of control plane have incalculable consequences. In open industrial networks, a number of adversaries and attacks are easier to affect safe communications. Thus, trustworthy and dependable consensus protocols are stringently required in SDIIoT.
- 3) With the widespread deployments of industrial networks, the limited scopes of these consensus protocols are challenging. A large-scale consensus protocol should be employed in SDIIoT.

These challenges have fueled the needs to explore new consensus protocols in SDIIoT. Inspire by the successful integration BC with secure key synchronization [31], and data sharing [32] in intelligent transportation systems, we consider that BC could be a potential approach to address the challenges in distributed SDIIoT. BC is a distributed system to provide dependable services to a group of nodes that do not fully trust each other. It can be seen as a third party system to achieve an agreement among nodes without a central trust broker, with the features of dependability, and in largely available system scale. In addition, considering the fact that distributed SDIIoT operates in partially trusted environment, and the advantages of permissioned BC, we utilize the permissioned BC in distributed SDIIoT.

III. SYSTEM MODEL

In this section, we introduce the system model that we use. We first present the network model, followed by the trust feature model and the computation model.

A. Network Model

We assume that there are C controllers in distributed SDIIoT, which are represented by $\mathcal{C} = \{1, \dots, C\}$. Each of them can communicate with the third-party BC system. This BC system consists of N nodes, i.e., physical machines, denoted by $\mathcal{N} = \{1, \dots, N\}$. Like other robust BFT protocols [33], these N nodes are under *Byzantine* failure model to make consensus, where at most f nodes are faulty [13], where $f = \lfloor (N-1)/3 \rfloor$. And any finite number of controllers can behave arbitrarily to issue correct or incorrect transactions to the BC system. Some strong adversaries can collude with each other to compromise the replicated service. However, they cannot break cryptographic technologies, i.e., signatures, message authentication code (MAC), and collision-resistant hashing.

We denote the messages with cryptographic technologies as follows [34].

- 1) $\langle m \rangle_{\sigma_i}$ means that message m is signed with a public key from node i .
- 2) $\langle m \rangle_{\sigma_{i,j}}$ means that message m is authenticated by node i with a MAC for node j .
- 3) $\langle m \rangle_{\sigma_i}$ means that message m is authenticated by an array of MACs with node i for every replicas.

Fig. 1 shows the different network structures between traditional scheme and BC-based scheme. It is worth mentioning that we use edge computing servers to do some computations related to the above cryptography so as to improve the throughput of BC system. There are E edge computing servers, and the set of computing servers is represented by $\mathcal{E} = \{1, 2, \dots, E\}$.

B. Trust Feature Model

We consider trust features of nodes and controllers in the system. Due to the lack of centralized security services and prior security association, all nodes and controllers have diverse trust features, such as safe or compromised. It is barely possible to exactly know what the trust feature is for one node or one controller at the next time instant. Thus, the trust features of a node $n \in \{1, 2, \dots, N\}$ and a controller $c \in \{1, 2, \dots, C\}$ can be modeled as random variables δ^n and η^c . δ^n and η^c can be divided into discrete levels, denoted by $\xi = \{\xi_0, \xi_1, \dots, \xi_{L-1}\}$ and $\mathcal{D} = \{\mathcal{D}_0, \mathcal{D}_1, \dots, \mathcal{D}_{H-1}\}$, respectively, where L and H are the number of available trust features for nodes and controllers. We assume the trust features realization of δ^n and η^c to be $\delta^n(t)$ and $\eta^c(t)$ at time slot t , respectively. There are T time slots during the period of time, which starts from when the controller issues an un-validated block, and terminates when the controller is replied with a validated block. Let $t \in \{0, 1, 2, \dots, T-1\}$ denote the time instant.

Considering the time correlation of real trust features in BC nodes and controllers, we use Markov chain to model the transition of trust features in BC nodes and controllers, as follows.

- 1) For node n , let the transition probability of $\delta^n(t)$ from one state $\mathcal{X}_s$ to another state $\mathcal{Y}_s$ be $\kappa_{\mathcal{X}_s \mathcal{Y}_s}(t)$. The $L \times L$ transition probability matrix $\mathcal{K}^n(t)$ of the trust feature in node n is denoted as

$$\mathcal{K}^n(t) = [\kappa_{\mathcal{X}_s \mathcal{Y}_s}(t)]_{L \times L} \quad (1)$$

where $\kappa_{\mathcal{X}_s \mathcal{Y}_s}(t) = \Pr(\delta^n(t+1) = \mathcal{Y}_s | \delta^n(t) = \mathcal{X}_s)$, and $\mathcal{X}_s, \mathcal{Y}_s \in \xi$.

- 2) For controller c , the transition probability of $\eta^c(t)$ is $\gamma_{\theta_s \phi_s}(t)$. The $H \times H$ transition probability matrix $\Upsilon^c(t)$ of the trust feature in controller c is denoted as

$$\Upsilon^c(t) = [\gamma_{\theta_s \phi_s}(t)]_{H \times H} \quad (2)$$

where $\gamma_{\theta_s \phi_s}(t) = \Pr(\eta^c(t+1) = \phi_s | \eta^c(t) = \theta_s)$ and $\theta_s, \phi_s \in \mathcal{D}$.

C. Computation Model

There are a number of computation tasks in the BC nodes, such as verifying signatures, generating MACs, and verifying

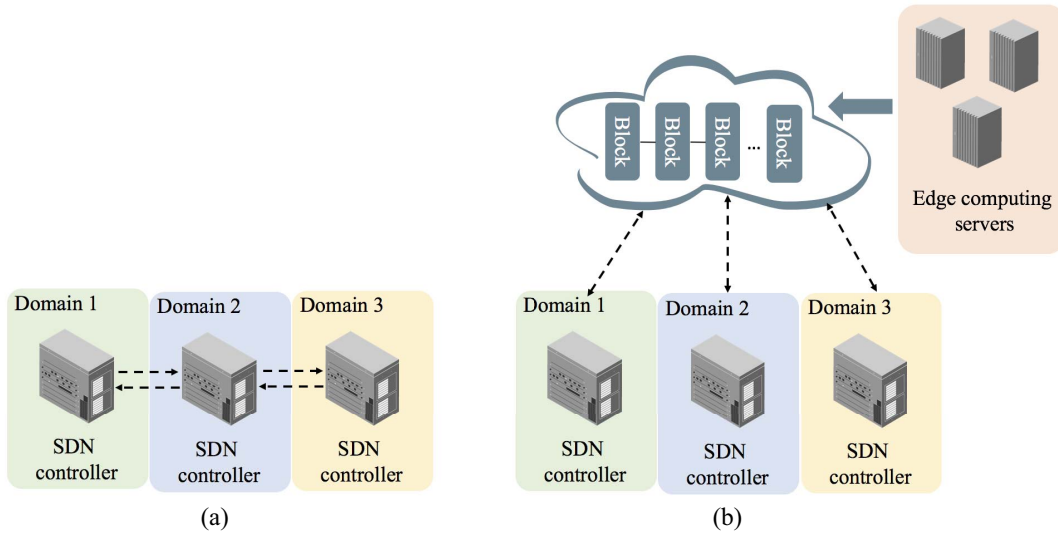


Fig. 1. Different network structures between traditional scheme and BC-based scheme.

MACs. Let $T_m = \{s_m, q_m\}$ denote a computation task related to message m , where s_m means the size of message m and q_m is the required number of CPU cycles to complete this task.

In order to improve the throughput of BC, we use virtual computing resources from edge computing servers to do computation tasks in the BC system. There are lots of edge computing servers and some other computation tasks that also use the computation resources in edge computing servers, thus we do not exactly know the computational resources for the BC system at the next time instant. Therefore, we model the computation resources of edge computing server e for the BC system as a random variable ζ^e . To discretize the values of computation capabilities, ζ^e can be partitioned into Y discrete intervals as $\mathcal{Y} = \{\mathcal{Y}_0, \mathcal{Y}_1, \dots, \mathcal{Y}_{Y-1}\}$. The computation resources of edge computing server e at time slot t can be denoted as $\zeta^e(t)$, $t \in \{0, 1, 2, \dots, T-1\}$. Considering the time correlation of computation state in edge computing server, we use a Markov chain to model the transition of computation state in edge computing server. Based on a certain transition probability, $\zeta^e(t)$ changes from one state to another. Let $\vartheta_{a_s b_s}(t)$ denote the transition probability. The $Y \times Y$ computation state transition probability matrix is represented as

$$\Pi^n(t) = [\vartheta_{a_s b_s}(t)]_{Y \times Y} \quad (3)$$

where $\vartheta_{a_s b_s}(t) = \Pr(\zeta^e(t+1) = b_s | \zeta^e(t) = a_s)$ and $a_s, b_s \in \mathcal{Y}$.

The execution time of computation task T_m can be denoted as

$$t_m = \frac{q_m}{\zeta^e(t)}. \quad (4)$$

Thus, the computation rate is

$$\text{CompR}^e(t) = a^e(t) \frac{s_m}{t_m} = a^e(t) \frac{\zeta^e(t) s_m}{q_m} \quad (5)$$

where $a^e(t)$ means whether or not edge computing server e is allocated to the BC system at time slot t . $a^e(t) = 1$ denotes edge computing server e is allocated to the BC

system; otherwise $a^e(t) = 0$. At one time slot, there is only one edge computing server allocated to the BC system, thus $\sum_{e=1}^E a^e(t) = 1$.

IV. BC-BASED CONSENSUS PROTOCOL

We have presented that the existing consensus protocols are challenging in SDIIoT, and BC could be a potential approach to address these issues in the previous section. In this section, we propose a novel BC-based consensus protocol in distributed SDIIoT. We begin with an overview of BC-based consensus protocol. Then, we present the detailed steps of the consensus protocol, along with theoretical analysis.

A. Overview of BC-Based Consensus Protocol

Each controller collects its local events and OpenFlow commands as Transaction #1, Transaction #2, ..., Transaction #n, which is called the collection period. The format of a transaction is shown in Table I. The number of this transaction denotes the position of this transaction. The signature and MAC make sure the integrity and authentication of this transaction. The payloads include local events and OpenFlow commands that need to be synchronized between different SDN controllers.

After the collection period, all controllers issue consensus requests to the third-party BC system. According to a policy, which will be introduced in the following section, called access selection, the BC system only enables one controller to access, and replies it by an admission message. Then, this controller sends an un-validated block with block header and transactions, whose format is presented in Table II. After reaching consensus, the BC system sends the corresponding validated block to the entire controllers. Finally, all controllers learn the payloads in each transaction to know the events and OpenFlow commands from other controller. These steps are in the consensus period. By this way, network-wide views can be synchronized between different SDN controllers.

TABLE I
 FORMAT OF A TRANSACTION

The number of this transaction in the block.
The Signature of this transaction.
The MAC of this transaction.
Payloads, including local events and OpenFlow commands.

 TABLE II
 FORMAT OF A BLOCK

Field	Description
Version	Block version number.
Timestamp	Creation time of this block.
Controller ID	The identifier of this controller.
Block ID	The identifier of this block.
Block payload	Transactions in this block (Transaction #1, ..., Transaction #n).

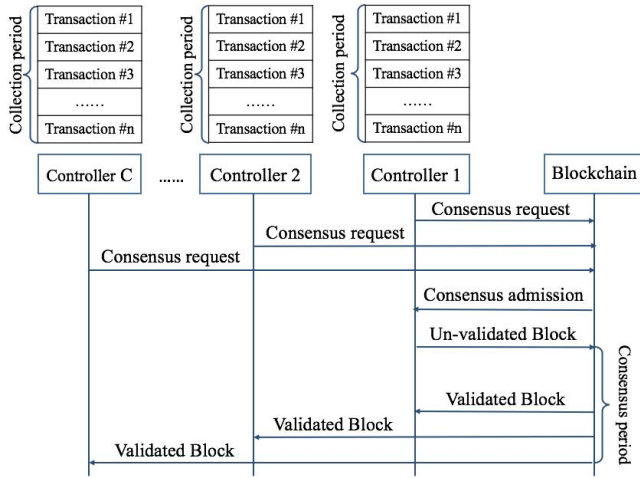


Fig. 2. Overview of consensus procedures in BC-based consensus protocol between different SDN controllers.

For a comprehensive perspective, Fig. 2 offers the consensus procedures in BC structure. Here, controller 1 is the selected controller to access to the BC system.

B. Detailed Steps and Theoretical Analysis

After giving the overview of BC-based consensus protocol, we will introduce the detailed steps inside the permissioned BC, along with the theoretical analysis of each step.

Based on PBFT [13], the detailed steps inside the permissioned BC is depicted in Fig. 3. The numbering of each step in this figure is the same as the one used in the remainder of this section. PBFT protocol has been used in real scenario, such as Hyperledger Fabric project [15], and Hyperledger Indy project [35]. They are hosted by The Linux Foundation, and develop applications with a modular architecture. Thus, we consider that it can be used in real scenario when meeting with SDN controllers.

1) *Controller Sends Unvalidated Block to All Nodes*: The selected controller sends an un-validated block to the BC system. The agent chooses one node in the BC as the primary node p . Making decisions about which node is the primary node is known as view change protocol. The view change

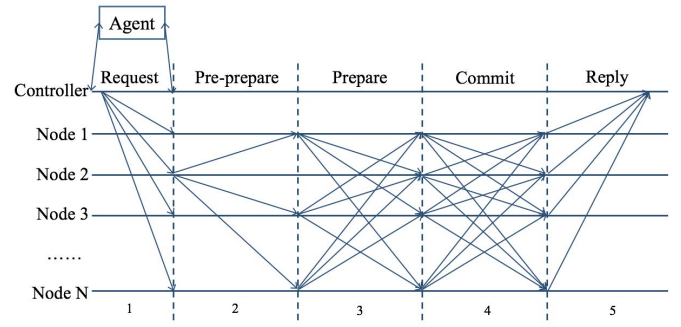


Fig. 3. Detailed procedures inside the permissioned BC.

protocol used in our proposed scheme will be introduced in the following section. This selected controller sends a block message $\langle \langle \text{block} \rangle_{\sigma_c}, c \rangle_{\sigma_c}$ to all nodes, where c denotes the controller ID. It is encrypted with the private signature of controller c , and authenticated with MACs for all nodes. When receiving this message, only primary node p verifies the MAC. If valid, the signature will be verified then. If still valid, it verifies the signature and the MAC of each transaction in this block, then moves to the following steps. The success rate of all verification will be recorded by the agent. If this block has already been executed, the primary will resend the validated block to this controller.

Theoretical Analysis: We consider an uncivil execution during which a fraction g of transactions sent by the controller are correct [34]. The more trusted controller has the bigger g . We assume verifying one signature, generating one MAC, and verifying one MAC require θ , α , and α cycles, respectively, and the controller issues batches of transactions of size b . As the work in [13], we ignore the cost of sending and receiving transactions. Therefore, the cost at the primary is

$$\left(1 + \frac{b}{g}\right)(\theta + \alpha) \quad (6)$$

and there is no cost at nonprimary nodes.

2) *Primary Node Multicasts PRE-PREPARE Message to Other Replica Nodes*: Finishing the verification, primary p sends a PRE-PREPARE message to all other replica nodes, as $\langle \text{PRE} - \text{PREPARE}, p, c, H(m) \rangle_{\sigma_p}$, which is authenticated with MACs for each replica node. Here, p and $H(m)$ mean the primary node ID and the hashed result of the issued block, respectively. The replica node verifies the MAC of primary p , as well as the signature and the MAC of each transaction. Then it enters the following steps.

Theoretical Analysis: In this phase, primary p generates $(N - 1)$ MACs for all replicas. Each replica verifies one MAC from primary p , as well as b/g signatures and MACs from transactions. Thus, the cost at the primary is

$$(N - 1)\alpha \quad (7)$$

and the cost at each replica node is

$$\alpha + \frac{b}{g}(\theta + \alpha). \quad (8)$$

3) *Replicas Send PREPARE Message to Others*: After verifying the validity of MACs and signatures, each replica replies

the PRE-PREPARE message with sending a PREPARE message to all nodes, as $\langle \text{PREPARE}, p, c, H(m), n \rangle_{\sigma_n}$, where n denotes the replica node ID. When each replica node collects $2f$ matching PREPARE messages with its local PRE-PREPARE message, it will enter the following steps.

Theoretical Analysis: In this phase, primary p needs to verify $2f$ MACs. Each replica node generates $(N-1)$ MACs and verifies $2f$ MACs. Therefore, the cost at the primary is

$$2f\alpha \quad (9)$$

and the cost at each replica is

$$(N-1+2f)\alpha. \quad (10)$$

4) *All Nodes Send COMMIT Message to Others:* Following the reception of $2f$ matching PREPARE messages, node n sends a COMMIT message to all others, as $\langle \text{COMMIT}, p, c, H(m), n \rangle_{\sigma_n}$. After receiving $2f$ matching COMMIT messages, it will enter the following steps.

Theoretical Analysis: In this phase, primary p needs to generate $(N-1)$ MACs, and verify $2f$ MACs. Each replica generates $(N-1)$ MACs, and verifies $2f$ MAC. Therefore, the costs at the primary and the replica are both

$$(N-1+2f)\alpha. \quad (11)$$

5) *Nodes Send the Validated Block to All Controllers:* Node n sends a REPLY message $\langle \text{REPLY}, \text{block}, n \rangle_{\sigma_{n,c}}$ to all controllers, where block is the validated block. When each controller receives $2f$ valid and matching REPLY messages, it accepts this validated block and updates the corresponding network views.

Theoretical Analysis: In this phase, the primary and the replicas need to generate b/g MACs for one controllers. Therefore, the total costs at the primary and the replica are both

$$\frac{b}{g}C\alpha. \quad (12)$$

Thus, for one transaction, the cost at the primary is

$$\left(\frac{1}{g} + \frac{1}{b}\right)\theta + \left(\frac{1}{b} + \frac{C+1}{g}\right)\alpha + \frac{2N+4f-2}{b}\alpha. \quad (13)$$

For one transaction, the cost at the replica is

$$\frac{1}{g}\theta + \frac{2+C}{g}\alpha + \frac{2N+4f-2}{b}\alpha. \quad (14)$$

We assume that multicore computation modules run in parallel on distinct cores. Each core has the computation speed of φ Hz. In addition, we consider an uncivil execution where the primary is not fully trusted. The primary has k trust to affect the system performance, and $k \in [0, 1]$. The more trusted primary node has the bigger k . Therefore, the throughput of the consensus protocol is at most

$$\min \left[\frac{k\varphi}{\left(\frac{1}{g} + \frac{1}{b}\right)\theta + \left(\frac{1}{b} + \frac{C+1}{g}\right)\alpha + \frac{2N+4f-2}{b}\alpha}, \frac{k\varphi}{\frac{1}{g}\theta + \frac{2+C}{g}\alpha + \frac{2N+4f-2}{b}\alpha} \right] \text{trx/s}. \quad (15)$$

V. PROBLEM FORMULATION

We have presented the throughput of the BC system. In order to improve the throughput, we need to make the joint decisions about view changes, access selection, and computational resources allocation. In this section, we formulate this joint problem as a Markov decision process by defining state space, action space, and reward function.

A. State Space

In order to improve the throughput, the learning agent needs to sense state $s(t)$ at time slot t . As we have mentioned, the learning agent should make the joint decisions about view changes, access selection, and computational resources allocation. Accordingly, the learning agent needs to sense the trust features of all nodes and controllers, as well as the computational capabilities of all edge computing servers. Therefore, the state space can be represented as follows:

$$s(t) = \begin{bmatrix} \delta^1(t) & \delta^2(t) & \dots & \delta^N(t) \\ \eta^1(t) & \eta^2(t) & \dots & \eta^C(t) \\ \zeta^1(t) & \zeta^2(t) & \dots & \zeta^E(t) \end{bmatrix}. \quad (16)$$

B. Action Space

The agent mainly needs to decide view changes (i.e., which node is the primary node), access selection (i.e., which controller can access to the BC system), and computational resources allocation (i.e., which edge computing server should be allocated to the BC system). Thus, the action space is denoted by

$$A(t) = \{A^N(t), A^C(t), A^E(t)\} \quad (17)$$

where $A^N(t)$, $A^C(t)$, and $A^E(t)$ represent the following.

- 1) $A^N(t) = [a^1(t), a^2(t), \dots, a^n(t), \dots, a^N(t)]$, which means whether or not node n is the primary. And $a^n(t) \in \{0, 1\}$, where $a^n(t) = 1$ means node n is the primary node, otherwise it is the replica node. Note that the BC system only has one primary node, thus $\sum_{n=1}^N a^n(t) = 1$.
- 2) $A^C(t) = [a^1(t), a^2(t), \dots, a^c(t), \dots, a^C(t)]$ decides which controller can access to the BC system. And $a^c(t) \in \{0, 1\}$, where $a^c(t) = 1$ represents controller c can access, otherwise $a^c(t) = 0$. Note that at one time slot, only one controller enables to access to the BC system, thus $\sum_{c=1}^C a^c(t) = 1$.
- 3) $A^E(t) = [a^1(t), a^2(t), \dots, a^e(t), \dots, a^E(t)]$ determines which edge computing server is allocated to the BC system. And $a^e(t) \in \{0, 1\}$, where $a^e(t) = 1$ denotes edge computing server e is allocated, otherwise $a^e(t) = 0$. Similarly, $\sum_{e=1}^E a^e(t) = 1$.

C. Reward Function

In order to improve the throughput, we model the throughput of the BC system as reward function. According to (15), state space, and action space, we define the

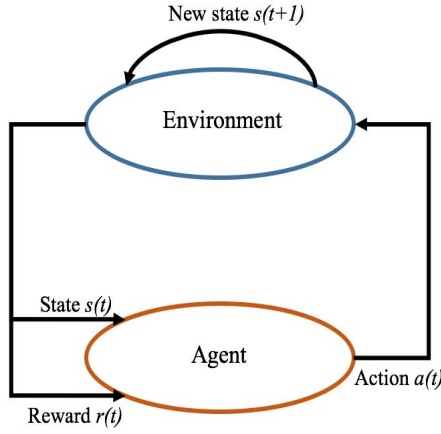


Fig. 4. Interaction of the learning agent and the environment.

reward function as

$$r(t) = \min \left[\frac{k' \varphi'}{\left(\frac{1}{g'} + \frac{1}{b}\right)\theta + \left(\frac{1}{b} + \frac{C+1}{g'}\right)\alpha + \frac{2N+4f-2}{b}\alpha}, \frac{k\varphi}{\frac{1}{g'}\theta + \frac{2+C}{g'}\alpha + \frac{2N+4f-2}{b}\alpha} \right] \text{tr}x/s \quad (18)$$

where $k' = \sum_{n=1}^N a^n(t)\delta^n(t)$, $\varphi' = \sum_{e=1}^E a^e(t)\text{Comp}R^e(t)$, and $g' = \sum_{c=1}^C a^c(t)\eta^c(t)$.

Based on the above problem formulation, the learning agent senses state $s(t)$ at time slot t , and outputs a policy π that determines which action $a(t)$ should be taken. Then this action will be executed, i.e., one controller will enable to access to the BC system, one node will be the primary node, and one edge computing server will be allocated to the BC system. In order to let the learning agent remember the experience and act better next time, the immediate reward $r(t)$ will be fed back to the learning agent. The trust features of nodes and controllers, as well as the computational capabilities of edge computing servers change to next state $s(t+1)$. Then the learning agent senses them and outputs another new policy, and so on and so forth. The final goal is to achieve the maximum long-term reward. Summarily, the interaction of the learning agent and the environment is shown in Fig. 4.

There are some challenges to solve the above problem formulation as follows.

- 1) The target is to maximize the long-term reward by step-and-step control. However, the learning agent only senses the state at time slot t , and the action taken at time slot t will affect the environment at time slot $t+1$. The state cannot be obtained ahead of time. Thus, the traditional optimization method, only considering current state, is not feasible.
- 2) Considering the trust features of nodes and controllers, as well as the computational capabilities of edge computing servers, the system is high-dimensional and high-dynamical. It is hard to make the joint and optimal decisions by traditional methods.
- 3) In the BC system, taking which action has no relationship with what happens in the next time slot. For

example, when the learning agent selects one controller to access to the BC system, its trust feature in the next time slot still changes according to its transition probability matrix, not the action. Therefore, the traditional optimization method, learning the relationship between states and actions, is not suitable.

To address the above challenges, we will propose a dueling DQL approach in the next section to achieve the maximum long-term reward.

VI. DUELING DEEP Q-LEARNING

After the problem formulation, we consider a dueling DQL approach to address this problem. In this section, we begin with the introduction of Q -learning and DQL. Then we present dueling DQL that is used in this paper.

A. Q -Learning

In the Q -learning model, the agent interacts with the environment by perceptions and actions. In one interaction step, the agent receives current state $s(t)$ from the environment, then selects an action $a(t)$ as the output, and the value of this action is measured by a scalar reward $r(t)$. This action generates next state $s(t+1)$. The agent selects actions to obtain the maximum long-term rewards. It learns to do this over several interaction steps by systematic trials and errors, guided by Q -learning [36]. Q -learning is a model-free algorithm using delay rewards. It aims to find a policy π , mapping states and actions, to maximize the long-term rewards.

There are two popular approaches to denote the feedback from each step in terms of long-term rewards, namely state-value function $V^\pi(s)$ and action-state value function $Q^\pi(s, a)$. $V^\pi(s)$ means the expected total reward in state s

$$V^\pi(s) = E^\pi \left[\sum_{k=1}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right] \quad (19)$$

where $E^\pi[\cdot]$ means mathematical expectation, r_{t+k+1} means the immediate reward at time slot $t+k+1$, and $\gamma \in (0, 1)$ is discount factor to balance immediate reward and future reward. Moreover, $Q^\pi(s, a)$ denotes the expected total rewards in state s and action a

$$Q^\pi(s, a) = E^\pi \left[\sum_{k=1}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a \right]. \quad (20)$$

Q -learning evaluates $Q(s, a)$ by a temporal difference method as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right) \quad (21)$$

where α is learning rate, and $\alpha \in (0, 1]$. The action with maximum $Q(s, a)$ may be chosen by the agent at each step. In the traditional Q -learning, each $Q(s, a)$ is put into Q -table. However, with the rapid increase of data dimension, it is challenging to put all $Q(s, a)$ into Q -table.

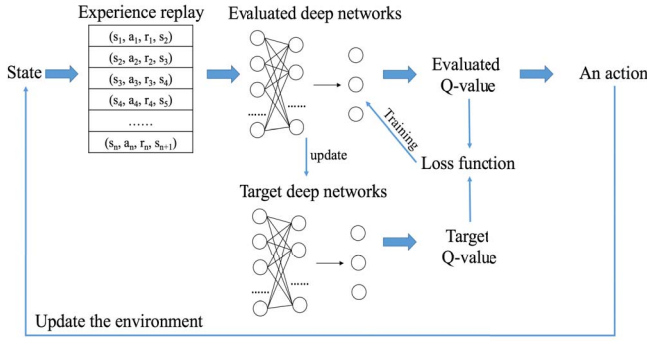


Fig. 5. Workflows of DQL.

B. Deep Q-Learning

The rise of deep learning has provided a new tool to overcome the challenges. The most important property of deep learning is that deep networks enable to find the low-dimensional features of high-dimensional data by crafting weights and biases in deep networks. Therefore, many researches have advocated to use deep networks to approximate $Q(s, a)$ instead of Q -table, i.e., $Q(s, a, \omega) \approx Q(s, a)$, where ω is the set of weights and biases in deep networks [37]. This is the core idea of DQL.

In order to address the fundamental instability problem of approximating $Q(s, a)$, there are two improvements in DQL, including experience replay and fixed target networks.

- 1) Experience replay stores the transitions as a set of $\{\text{state}, \text{action}, \text{reward}, \text{state}_{\text{next}}\}$ in a finite-sized cyclic buffer, and the agent randomly samples batches of them to train deep networks, instead of only the current ones. By this way, the temporal correlations that can adversely affect DQL are broken.
- 2) Fixed target networks have the same architecture as the evaluated ones, but are kept frozen for a period of time.

The evaluated networks are trained in each step to minimize loss function $L(\omega)$ to evaluate real $Q(s, a)$, and $L(\omega)$ is represented as

$$L(\omega) = E \left[\left(r + \gamma \max_{a'} Q(s', a', \omega^-) - Q(s, a, \omega) \right)^2 \right] \quad (22)$$

where ω^- is the weights and biases set in target networks and ω is the weights and biases set in evaluated networks. During training, the weights and biases in target networks are updated with evaluated networks periodically. For a comprehensive perspective, we present the workflows of DQL in Fig. 5.

C. Dueling Deep Q-Learning Approach

However, for the majority of states in our system, the choice of actions in the agent has no repercussion with what happens, i.e., actions have no relationship with states. According to the work in [38], dueling DQL is more efficient than natural DQL. Based on this approach, some training processes have been

Algorithm 1 Dueling DQL

- 1: **Initialization:**
Initialize evaluated deep networks with weights and biases set ω .
Initialize target deep networks with weights and biases set ω^- .
- 2: **for** $k = 1 : K$ **do**
- 3: Reset the environment with a randomly initial observation s_{ini} , and $s(t) = s_{\text{ini}}$.
- 4: **while** $s(t) \neq s_{\text{terminal}}$ **do**
- 5: Select action $a(t)$ based on ϵ -greedy policy.
- 6: Obtain immediate reward $r(t)$ and next observation $s(t+1)$.
- 7: Store experience $(s(t), a(t), r(t), s(t+1))$ into experience replay memory.
- 8: Randomly sample some batches of $(s(i), a(i), r(i), s(i+1))$ from experience replay memory.
- 9: Calculate two streams of evaluated deep networks, including $V(s; \omega, \varrho)$ and $A(s, a; \omega, \zeta)$, and combine them as $Q(s, a; \omega, \varrho, \zeta)$ using (25).
- 10: Calculate target Q-value $Q_{\text{target}}(s)$ in target deep networks:
 if s' is s_{terminal}
 $Q_{\text{target}}(s) = r_s$,
 else
 $Q_{\text{target}}(s) = r_s + \gamma \max_{a'} Q(s', a'; \omega', \varrho', \zeta')$.
- 11: Train evaluated deep networks to minimize loss function $L(\omega)$
- 12: Every some steps, update target deep networks.
- 13: $s(t) \leftarrow s(t+1)$
- 14: **end while**
- 15: **end for**

$$L(\omega, \varrho, \zeta) = E \left[\left(Q_{\text{target}}(s) - Q(s, a; \omega, \varrho, \zeta) \right)^2 \right]. \quad (23)$$

carried out in real scenario, such as drive games [38], which are simulated with better performance.

In the dueling DQL, there is another value function, $A(s, a)$, which represents the relative advantage of a action, called state-action value function. Learning $A(s, a)$ is easier to know which action has better consequences. Instead of one single stream following the output layer of deep networks, there are two separate streams in dueling DQL, where one computes state-value function $V(s)$, and another computes state-action value function $A(s, a)$, called dueling architecture as shown in Fig. 6. Finally, these two streams are aggregated as a output $Q(s, a)$. This combination module is denoted as follows:

$$Q(s, a; \omega, \varrho, \zeta) = V(s; \omega, \varrho) + A(s, a; \omega, \zeta) \quad (24)$$

where $V(s; \omega, \varrho)$ is a scalar, and $A(s, a; \omega, \zeta)$ is an $|\mathcal{A}|$ -dimensional vector. ϱ and ζ are the parameters of two separate streams.

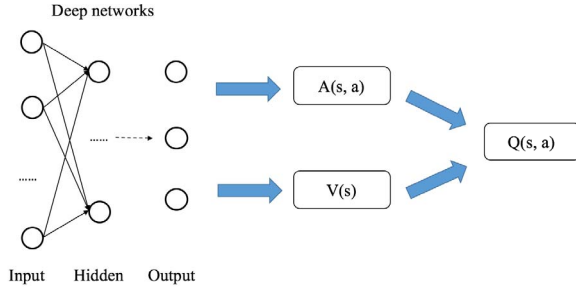


Fig. 6. Dueling architecture.

According to the work in [38], considering the unidentifiability of (24), we replace (24) as

$$\begin{aligned} Q(s, a; \omega, \varrho, \zeta) \\ = V(s; \omega, \varrho) + \left(A(s, a; \omega, \zeta) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \omega, \zeta) \right). \end{aligned} \quad (25)$$

As the output of dueling architecture is $Q(s, a)$, it can be trained by many existing algorithms. In this paper, we use dueling architecture in natural DQL. Therefore, the pseudocode of dueling DQL is presented in Algorithm 1, where ϵ -greedy policy is used to balance the exploitation and the exploration.

VII. SIMULATION RESULTS AND DISCUSSION

In this section, we use computer simulation to evaluate the performance of our proposed scheme. First, we present simulation settings, followed by simulation results and the corresponding discussions.

A. Simulation Settings

In this simulation, hardware environment is a GPU-based server, and this server has 8-GB 1867-MHz LPDDR3, 2-GHz Intel Core i5, and 256-GB memory. Software environment is Python 2.7.10 with TensorFlow 1.4.0 [39]. These two kinds of simulation tools have been widely used from business to academic. TensorFlow is a flexible, high-performance serving system, used by machine learning models, and designed for production environments. TensorFlow enables to keep the same server architecture and application program interface, when different machine learning algorithms are deployed. Thus, it has been widely used to deploy new machine learning algorithms and experiments. We use these real simulation environments, thus we can make sure that the performance of simulation results can estimate and approximate the performance in real scenario.

We assume that there are four BC nodes, two controllers, and two edge computing servers. The trust feature of each node is very safe, safe, medium, compromised, and very

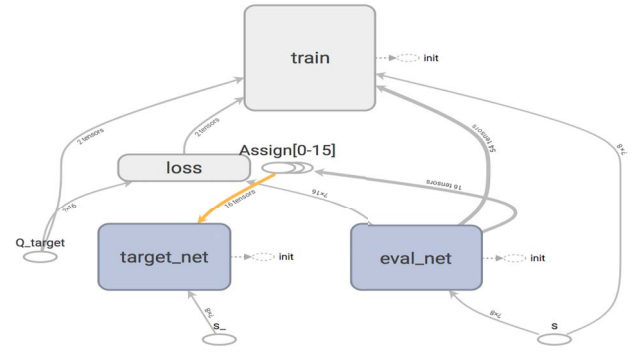


Fig. 7. Visualized TensorFlow graph in TensorBoard.

compromised, whose transition probability matrix is

$$\mathcal{K} = \begin{bmatrix} 0.5 & 0.15 & 0.125 & 0.12 & 0.105 \\ 0.15 & 0.5 & 0.125 & 0.12 & 0.105 \\ 0.105 & 0.15 & 0.5 & 0.125 & 0.12 \\ 0.105 & 0.12 & 0.125 & 0.5 & 0.15 \\ 0.105 & 0.12 & 0.125 & 0.15 & 0.05 \end{bmatrix}. \quad (26)$$

Similarly, the trust feature of each controller can be very safe, safe, medium, compromised, and very compromised. We set the transition probability matrix as

$$\Upsilon = \begin{bmatrix} 0.45 & 0.16 & 0.14 & 0.13 & 0.12 \\ 0.16 & 0.45 & 0.14 & 0.13 & 0.12 \\ 0.12 & 0.16 & 0.45 & 0.14 & 0.13 \\ 0.12 & 0.13 & 0.16 & 0.45 & 0.14 \\ 0.12 & 0.13 & 0.14 & 0.16 & 0.45 \end{bmatrix}. \quad (27)$$

We assume the computational capability of each edge computing server is high, medium, low, and very low, and set the transition probability as

$$\Pi = \begin{bmatrix} 0.5 & 0.3 & 0.15 & 0.05 \\ 0.3 & 0.5 & 0.15 & 0.05 \\ 0.15 & 0.3 & 0.5 & 0.05 \\ 0.15 & 0.3 & 0.5 & 0.05 \end{bmatrix}. \quad (28)$$

The values of the rest of parameters are summarized in Table III. We use TensorBoard to visualize the TensorFlow graph, as shown in Fig. 7.

For the performance comparison, there are four schemes simulated.

- 1) Proposed dueling DQL-based scheme with view changes, access selection, and edge computing servers. We call it dueling DQL-based scheme in the remainder of this section. In other words, in dueling DQL-based scheme, the learning agent enables to select more trusted BC node as the primary, more trusted controller to access to the BC system, and edge computing server with more computing capabilities. Thus, this scheme should have the best performance.
- 2) Proposed dueling DQL-based scheme with view changes, edge computing servers, but without access selection. We call it dueling DQL-based scheme without controller choice in the remainder of this section. That is to say, in this scheme, the learning agent only enables to select more trusted BC node as the primary, and edge

TABLE III
PARAMETERS SETTING IN THE SIMULATION

Parameter	Value	Description
θ	8 Mcycles	The required number of CPU cycles to verify one signature.
α	0.05 Mcycles	The required number of CPU cycles to verify and generate one MAC.
b	1Mb	The batch size of a block.
γ	0.9	The discount factor.

computing server with more computing capabilities, but controllers access to the BC system randomly. Thus, some malicious controllers may issue incorrect transactions to slow down the performance. Compared with the dueling DQL-based scheme, the advantage of selecting trusted controller can be indicated.

- 3) Proposed dueling DQL-based scheme with access selection, edge computing servers, but with the traditional view change protocol in [13]. We call it dueling DQL-based scheme without node choice in the remainder of this section. In other words, in this scheme, the learning agent only selects more trusted controller to access to the BC system, and edge computing server with more computing capabilities, but using the traditional view change protocol without considering historical trust reputation. Thus, some malicious nodes may be selected as the primary to weaken the throughput. Compared with the dueling DQL-based scheme, the merit of selecting trusted BC node can be shown.
- 4) Proposed dueling DQL-based scheme with view changes, access selection, but without edge computing servers, which only uses the local computing capabilities in the BC system. We call it dueling DQL-based scheme without computation offloading in the remainder of this section. That is to say, the learning agent only selects more trusted BC node as the primary, and more trusted controller to access to the BC system, but only using the local computing capabilities. Without the assistance of edge computing servers, it is difficult to do cryptographic tasks only by local computation, which may slow down the performance. Compared with the dueling DQL-based scheme, the advantage of using edge computing servers can be indicated.
- 5) Existing scheme with traditional view changes, without access selection, and with local computational capabilities. We call it existing scheme in the remainder of this section. The dueling DQL approach is not employed in this scheme. This scheme allows controllers to access to the BC system randomly, uses the traditional view change protocol and local computing capabilities. Compared with the dueling DQL-based scheme, the advantage of using the dueling DQL approach can be shown.

B. Simulation Results

Fig. 8 shows the relationship between training episodes and the throughput of the BC system under different schemes. Each point is the average throughput per episode. The agent runs in AdamOptimizer [40] with the learning rate of $1e^{-5}$. As we

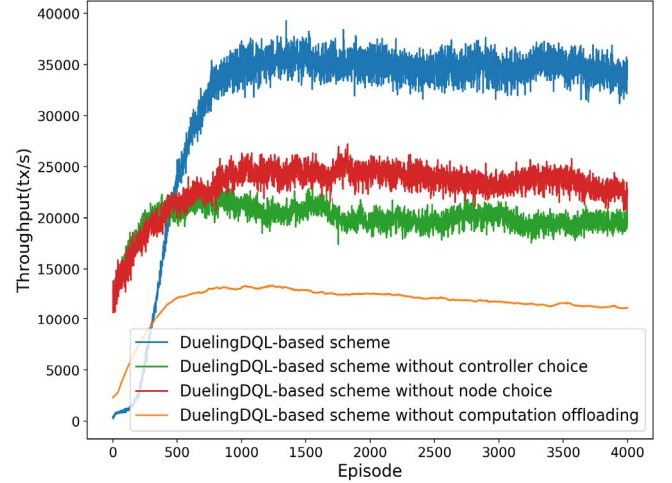


Fig. 8. Training curves tracking the throughput of the BC system under different schemes.

can see from this figure, with the joint consideration of node's trust feature, controller's trust feature, and offloading the computation task to edge computing servers, the BC system has the better performance. The reason is that the more trusted node is less possible to slow the system performance down, the more trusted controller issues the higher fraction of correct transactions, and with the help of edge computing servers, computation tasks can be executed more quickly. This figure shows the convergence performance of dueling DQL as well. At the beginning of learning and training, dueling DQL takes some trials and errors. With the increase of episodes, the throughput turns to be stable, which means the agent has learned the optimal policies to maximize the long-term rewards.

In addition, Fig. 9 shows the relationship between the learning loss in (22) and the training steps of dueling DQL-based scheme, when the agent runs in the same parameters as above. At the beginning of learning, deep networks have no knowledge of the uncertain environment, and with the increase of new experiences, the learning loss is higher and higher. When the cyclic buffer of experiences in dueling DQL is full, the agent has some knowledge of the environment, which leads to the decrease of the learning loss. Such increasing and decreasing of the learning loss indicate the effectiveness of deep networks.

Fig. 10 shows the relationship between training episodes and the throughput under different learning rates in dueling DQL-based scheme. As we can see from this figure, the learning rate has effects on the convergence performance. The learning rate means the length of learning step to minimize the loss

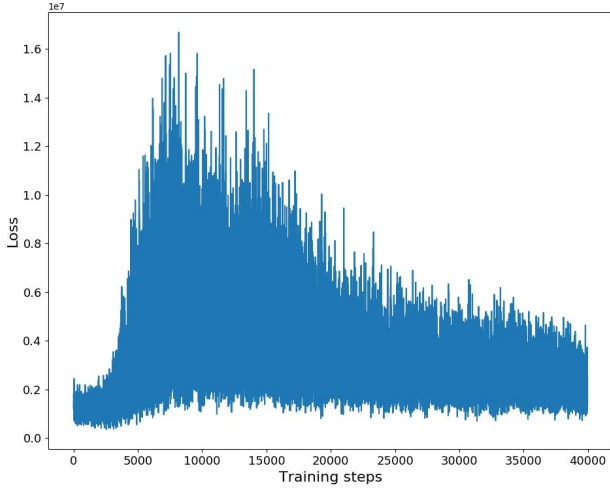


Fig. 9. Training curves tracking the learning loss under dueling DQL-based scheme.

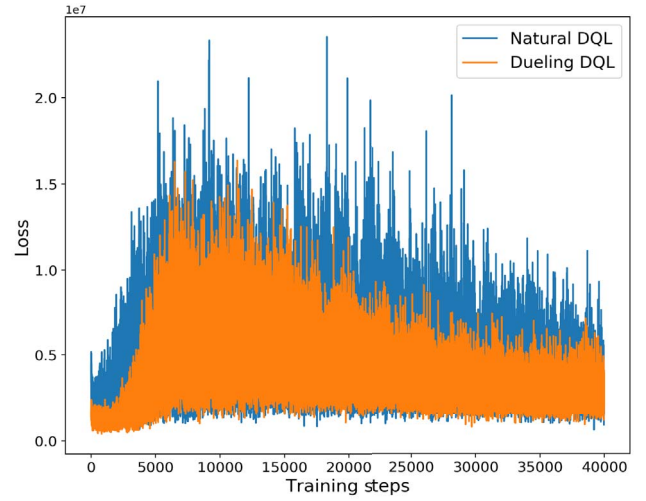


Fig. 11. Training curves tracking the learning loss of natural DQL and dueling DQL.

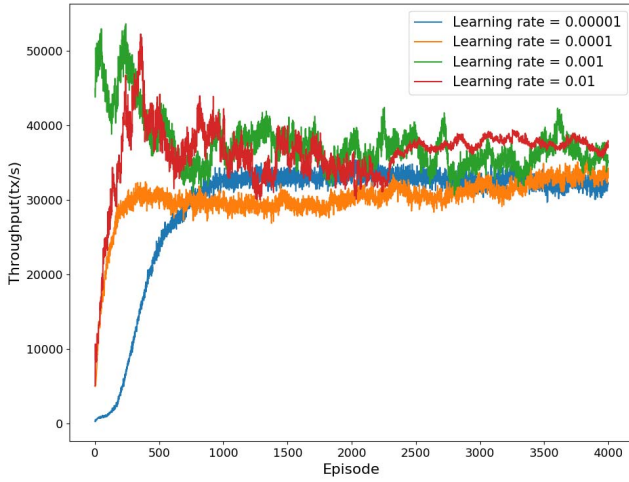


Fig. 10. Training curves tracking the throughput of the BC system under different learning rates.

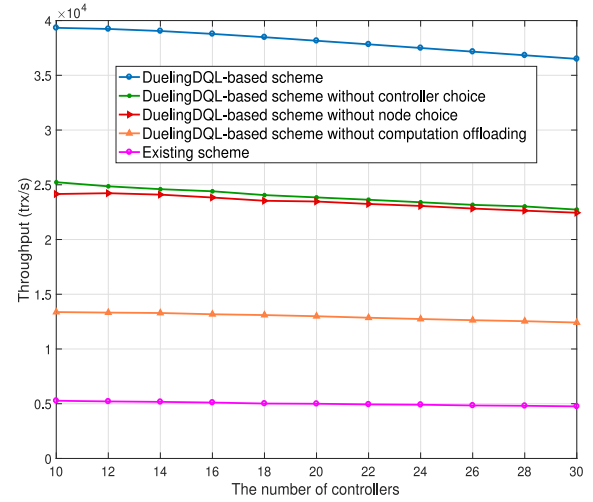


Fig. 12. Throughput versus the number of controllers under different schemes.

function. The bigger learning rate denotes the longer learning step. The longer learning steps are likely to miss the global optimum, which leads to the highly scaled curves when learning rates are 0.01 and 0.001. The shorter learning steps may lead to the slower convergence speed, because more steps are necessary to achieve the global optimum. Compared with two curves of blue and orange, although the orange one has the faster convergence speed, its curve is unstable after the convergence. Therefore, we choose the learning rate as $1e^{-5}$ in the simulation. Because its convergence speed is acceptable and it has better learning stability.

Fig. 11 shows the learning loss of natural DQL and dueling DQL. As we can see, the learning loss in dueling DQL decreases more quickly than natural DQL, which indicates dueling DQL has better learning effectiveness. The reason is that in our BC system, the choices of which node is the primary, which controller can access to the BC system, and which edge computing server should execute the computation tasks have no relationship with states. Learning which action has

better consequence is more efficient than learning which state is better. In dueling DQL, one stream learns state-action value function $A(s, a)$, which is more useful to help the agent make the good choices. Therefore, the learning loss in dueling DQL decreases more fast than natural DQL.

After the effective training of deep networks, we use them in the following simulations. Fig. 12 shows the relationship between the number of controllers and the system throughput under different schemes. This figure also can be used to show the performance of these schemes in large real SDIIoT environment, where up to 30 controllers are considered in this simulation. As the increase of the number of controllers, the throughput of the BC system decreases. The reason is that more controllers need more computational operations about verifying signatures, MACs. But with the joint consideration of trust features of controllers and nodes, as well as using edge computing servers, our proposed scheme, as shown in the blue curve, has better performance. Thus, we can see our proposed scheme still has better performance in large SDIIoT environment.

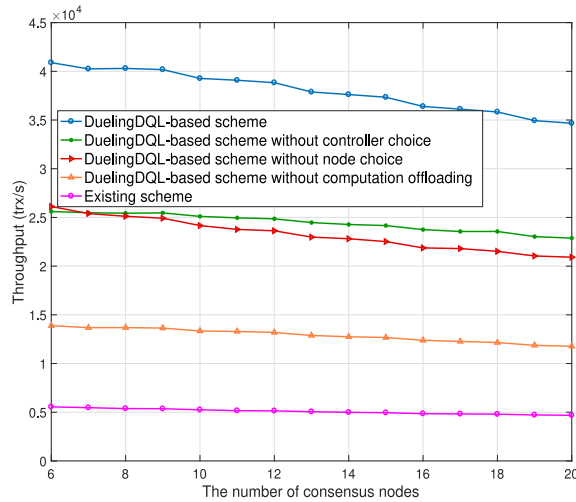


Fig. 13. Throughput versus the number of BC nodes under different schemes.

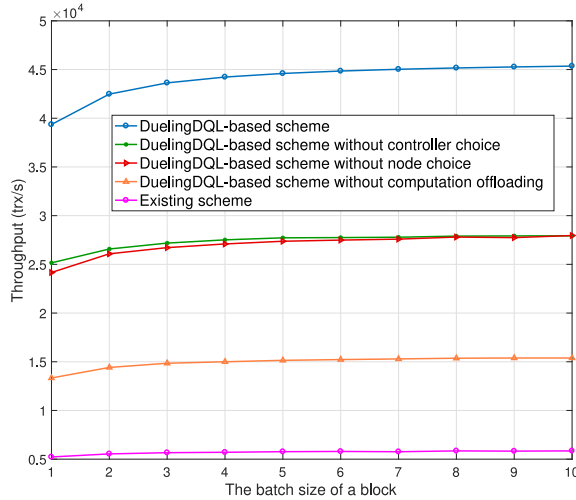


Fig. 14. Throughput versus the batch size of a blocks under different schemes.

Fig. 13 shows the relationship between the number of BC nodes and the system throughput under different schemes. As we can see, more nodes lead to the less system throughput. The reason is that with the increase of the number of nodes, more signatures and MACs need to be verified and generated, which need more CPU cycles so as to decrease the system throughput. But, the performance of our proposed scheme is still the best.

Fig. 14 shows the relationship between the batch size of a block and the system throughput. The bigger block enables to contain more transactions so as to synchronize more local network events among controllers, which increases the system throughput. As we can see, our scheme also has the better performance.

VIII. CONCLUSION

In this paper, we proposed a BC-based consensus protocol in distributed SDIIoT, along with detailed consensus steps and theoretical analysis, where BC acted as a trusted third party to collect and synchronize network-wide views between

different SDN controllers. In order to improve the throughput of the BC system, we jointly considered the trust features of BC nodes and controllers, as well as the computational capability of the system. Accordingly, we formulated view changes, access selection, and computational resources allocation as a joint optimization problem. Due to the fact that it is difficult to solve the joint problem by traditional methods, we proposed a novel dueling DQL approach to solve this problem. Simulation results showed the effectiveness and the convergence performance of our proposed scheme with different scenarios. How to measure the trust features of nodes and controllers in SDIIoT is very important. Some future works are in progress to solve this problem.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their detailed reviews and constructive suggestions, which have helped to greatly improve the quality of this paper.

REFERENCES

- [1] J.-Q. Li *et al.*, "Industrial Internet: A survey on the enabling technologies, applications, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1504–1526, 3rd Quart., 2017.
- [2] L. Cui, F. R. Yu, and Q. Yan, "When big data meets software-defined networking: SDN for big data and big data for SDN," *IEEE Netw.*, vol. 30, no. 1, pp. 58–65, Jan./Feb. 2016.
- [3] J. Wan *et al.*, "Software-defined Industrial Internet of Things in the context of industry 4.0," *IEEE Sensors J.*, vol. 16, no. 20, pp. 7373–7380, Oct. 2016.
- [4] H. R. Faragardi, H. Fotuhi, T. Nolte, and R. Rahmani, "A cost efficient design of a multi-sink multi-controller WSN in a smart factory," in *Proc. Conf. High Perform. Comput. Commun.*, 2017, pp. 594–602.
- [5] K. Kaur *et al.*, "Edge computing in the Industrial Internet of Things environment: Software-defined-networks-based edge-cloud interplay," *IEEE Commun. Mag.*, vol. 56, no. 2, pp. 44–51, Feb. 2018.
- [6] N. G. Nayak, F. Dürr, and K. Rothermel, "Incremental flow scheduling and routing in time-sensitive software-defined networks," *IEEE Trans. Ind. Informat.*, vol. 14, no. 5, pp. 2066–2075, May 2018.
- [7] J. Wang, C. Jiang, Z. Han, Y. Ren, and L. Hanzo, "Network association strategies for an energy harvesting aided super-WiFi network relying on measured solar activity," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3785–3797, Dec. 2016.
- [8] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*. Accessed: Aug. 2018. [Online]. Available: <http://bitcoin.org/bitcoin.pdf>
- [9] F. R. Yu, J. Liu, Y. He, P. Si, and Y. Zhang, "Virtualization for distributed ledger technology (vDLT)," *IEEE Access*, vol. 6, pp. 25019–25028, 2018.
- [10] Netis-Group, *The Difference Between Permissionless and Permissioned Networks*. Accessed: Aug. 2018. <https://medium.com/netis-group-blog/>
- [11] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project, Yellow Paper, 2014.
- [12] C. Cachin and M. Vukolić, "Blockchains consensus protocols in the wild," *arXiv preprint arXiv:1707.01873*, pp. 1873–1897, 2017.
- [13] M. Castro and B. Liskov, "Practical Byzantine fault tolerance and proactive recovery," *ACM Trans. Comput. Syst.*, vol. 20, no. 4, pp. 398–461, 2002.
- [14] L. Lamport *et al.*, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, 2001.
- [15] C. Cachin, "Architecture of the hyperledger blockchain fabric," in *Proc. Workshop Distrib. Cryptocurrencies Consensus Ledgers*, 2016, pp. 121–125.
- [16] C. Jiang, Y. Chen, Y.-H. Yang, C.-Y. Wang, and K. R. Liu, "Dynamic Chinese restaurant game: Theory and application to cognitive radio networks," *IEEE Trans. Wireless Commun.*, vol. 13, no. 4, pp. 1960–1973, Apr. 2014.
- [17] J. Wang *et al.*, "Learning-aided network association for hybrid indoor LiFi-WiFi systems," *IEEE Trans. Veh. Technol.*, vol. 67, no. 4, pp. 3561–3574, Apr. 2018.

- [18] C. Jiang, Y. Chen, Y. Gao, and K. J. R. Liu, "Indian buffet game with negative network externality and non-Bayesian social learning," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 4, pp. 609–623, Apr. 2015.
- [19] C. Qiu, F. R. Yu, F. Xu, H. Yao, and C. Zhao, "Permissioned blockchain-based distributed software-defined Industrial Internet of Things," in *Proc. Globecom Workshops (GC Wkshps)*, 2018, pp. 1–7.
- [20] J. Wan *et al.*, "Toward dynamic resources management for IoT-based manufacturing," *IEEE Commun. Mag.*, vol. 56, no. 2, pp. 52–59, Feb. 2018.
- [21] M. Moness, A. M. Moustafa, A.-R. H. Muhammad, and A.-S. A. Younis, "Hybrid controller for a software-defined architecture of industrial Internet lab-scale process," in *Proc. Conf. Comput. Eng. Syst.*, 2017, pp. 266–271.
- [22] L. Silva, P. Gonçalves, R. Marau, P. Pedreiras, and L. Almeida, "Extending OpenFlow with flexible time-triggered real-time communication services," in *Proc. Conf. Emerg. Tech. Factory Autom.*, 2017, pp. 1–8.
- [23] Y. Duan, W. Li, Y. Zhong, and X. Fu, "A multi-network control framework based on Industrial Internet of Things," in *Proc. Conf. Netw. Sens. Control*, 2016, pp. 1–5.
- [24] F. Bannour, S. Souihi, and A. Mellouk, "Distributed SDN control: Survey, taxonomy, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 1, pp. 333–354, 1st Quart., 2018.
- [25] C. Qiu *et al.*, "A novel QoS-enabled load scheduling algorithm based on reinforcement learning in software-defined energy Internet," *Future Gener. Comput. Syst.*, vol. 1016, pp. 23–33, Sep. 2018.
- [26] A. Tootoonchian and Y. Ganjali, "HyperFlow: A distributed control plane for OpenFlow," in *Proc. Conf. Internet Netw. Manag.*, 2010, p. 3.
- [27] T. Koponen *et al.*, "Onix: A distributed control platform for large-scale production networks," in *Proc. OSDI*, vol. 10, 2010, pp. 1–6.
- [28] P. Berde *et al.*, "ONOS: Towards an open, distributed SDN OS," in *Proc. Conf. Hot Topics Softw. Defined Netw.*, 2014, pp. 1–6.
- [29] C. Qiu, C. Zhao, F. Xu, and T. Yang, "Sleeping mode of multi-controller in green software-defined networking," *EURASIP J. Wireless Commun. Netw.*, vol. 2016, no. 1, p. 282, 2016.
- [30] H. Yao, C. Qiu, C. Zhao, and L. Shi, "A multicontroller load balancing approach in software-defined wireless networks," *Int. J. Distrib. Sensor Netw.*, vol. 11, no. 10, 2015, Art. no. 454159.
- [31] A. Lei *et al.*, "Blockchain-based dynamic key management for heterogeneous intelligent transportation systems," *IEEE Internet Things J.*, vol. 4, no. 6, pp. 1832–1843, Dec. 2017.
- [32] M. Singh and S. Kim, "Blockchain based intelligent vehicle data sharing framework," *arXiv preprint arXiv:1708.09721*, pp. 7442–7446, 2017.
- [33] P.-L. Aublin, S. B. Mokhtar, and V. Quéma, "RBFT: Redundant Byzantine fault tolerance," in *Proc. Conf. Distrib. Comput. Syst.*, 2013, pp. 297–306.
- [34] A. Clement, E. L. Wong, L. Alvisi, M. Dahlin, and M. Marchetti, "Making Byzantine fault tolerant systems tolerate Byzantine faults," in *Proc. NSDI*, vol. 9, 2009, pp. 153–168.
- [35] *Hyperledger Indy*. Accessed: Aug. 2018. [Online]. Available: <https://cn.hyperledger.org/projects/hyperledger-indy>
- [36] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.
- [37] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.
- [38] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, pp. 6581–6596, 2015.
- [39] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, p. 4467, 2016.
- [40] T. M. Chilimbi, Y. Suzue, J. Apacible, and K. Kalyanaraman, "Project Adam: Building an efficient and scalable deep learning training system," in *Proc. OSDI*, vol. 14, 2014, pp. 571–582.

Chao Qiu (GS'18) received the B.S. degree in communication engineering from China Agricultural University, Beijing, China, in 2013. She is currently pursuing the Ph.D. degree at the Beijing University of Posts and Telecommunications, Beijing.

She is currently a visiting Ph.D. student with Carleton University, Ottawa, ON, Canada, supported by the China Scholarship Council. Her current research interests include big data, software-defined networking, and machine learning.

F. Richard Yu (S'00–M'04–SM'08–F'18) received the Ph.D. degree in electrical engineering from the University of British Columbia (UBC), Vancouver, BC, Canada, in 2003.

From 2002 to 2006, he was with Ericsson, Lund, Sweden, and a start-up in California, USA. He joined Carleton University, Ottawa, ON, Canada, in 2007, where he is currently a Professor. His current research interests include cross-layer/cross-system design, security, green IT, QoS provisioning in wireless-based systems, machine learning, and blockchain.

Dr. Yu was a recipient of the IEEE Outstanding Leadership Award in 2013, the Carleton Research Achievement Award in 2012, the Ontario Early Researcher Award (formerly the Premiers Research Excellence Award) in 2011, the Excellent Contribution Award of IEEE/IFIP TrustCom 2010, the Leadership Opportunity Fund Award from the Canada Foundation of Innovation in 2009, and the Best Paper Award of IEEE ICC 2014, Globecom 2012, IEEE/IFIP TrustCom 2009, and the International Conference on Networking 2005. He serves on the Editorial Boards of several journals, including as the Co-Editor-in-Chief for *Ad Hoc and Sensor Wireless Networks*, a Lead Series Editor for the IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, the *EURASIP Journal on Wireless Communications Networking*, the *Journal on Security and Communication Networks* (Wiley), and the *International Journal of Wireless Communications and Networking*. He has served as the Technical Program Committee Co-Chair for numerous conferences. He is a registered Professional Engineer in the Province of Ontario, Canada.

Haipeng Yao (M'16) received the Ph.D. degree from the Department of Telecommunication Engineering, University of Beijing University of Posts and Telecommunications, Beijing, China, in 2011.

He is an Associate Professor with the Beijing University of Posts and Telecommunications. He has authored or co-authored over 80 papers in prestigious peer-reviewed journals and conferences. His current research interests include future Internet architecture, network AI, big data, cognitive radio networks, and optimization of protocols and architectures for broadband wireless networks.

Chunxiao Jiang (S'09–M'12–SM'15) received the B.S. degree (Highest Hons.) in information engineering from the Beijing University of Aeronautics and Astronautics (Beihang University), Beijing, China, in 2008, and the Ph.D. degree (Highest Hons.) from Tsinghua University (THU), Beijing, in 2013.

From 2011 to 2012, he visited the Signals and Information Group, Department of Electrical and Computer Engineering (ECE), University of Maryland (UMD) at College Park, College Park, MD, USA, supported by the China Scholarship Council for one year. He is currently a Research Associate with the ECE Department, UMD, along with Prof. K. J. R. Liu, and also a Post-Doctoral Fellow with the EE Department, THU, along with Prof. Y. Ren. His current research interests include the applications of game theory and queuing theory in wireless communication and networking, and social networks.

Dr. Jiang was a recipient of the Best Paper Award of IEEE GLOBECOM in 2013, the Beijing Distinguished Graduated Student Award, the Chinese National Fellowship, and the Tsinghua Outstanding Distinguished Doctoral Dissertation in 2013.

Fangmin Xu received the M.S. and Ph.D. degrees in communication engineering from the Beijing University of Posts and Telecommunication (BUPT), Beijing, China, in 2003 and 2008, respectively.

He is currently an Associate Professor with the School of Information and Communications Engineering, BUPT. From 2008 to 2014, he was with Samsung Electronics, Seoul, South Korea, where he actively contributed to 3GPP LTE/LTE-A and IEEE 802.16m. He has authored 2 books, 20 peer-reviewed international research papers, and 50 standard contributions, and is the inventor of 15 issued or pending patents, among which 4 have been adopted in the specifications of 4G (3GPP LTE/LTE-A and IEEE 802.16m) standards. His current research interest includes advance technologies in wireless networks, especially the Internet of Things field.

Chenglin Zhao received the bachelor's degree in radio-technology from Tianjin University, Tianjin, China, in 1986, and the master's degree in circuits and systems and Ph.D. degree in communication and information system from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 1993 and 1997, respectively.

He currently serves as a Professor with BUPT. His current research interests include emerging technologies of short-range wireless communication, cognitive radios, and 60-GHz millimeter-wave communications.